OPERATING SYSTEMS

VTU Exam-Oriented Questions and Answers

Complete study compilation
Definitions • Explanations • Comparison tables • Algorithms • Formulae • Diagrams

Prepared from the questions and answers discussed in this chat. Repeated or overlapping questions are consolidated so that each topic appears once in a complete form.

Study format: concise, structured answers suitable for writing in a university examination.

Contents

Part I - Operating System Foundations
1. Operating System Services - User convenience and efficient system operation
2. Virtual Machines - Concept, hypervisors, advantages and challenges
3. Operating System Structures - Layered, modular and microkernel approaches
4. User Mode and Kernel Mode - Characteristics and mode switching
5. System Calls - Role, types, implementation and parameter passing
6. Inter-Process Communication - Shared memory and message passing
Part II - Processes and CPU Management
7. Process, States and PCB - Process states, transitions and PCB structure
8. Process Creation and Termination - Creation steps, termination mechanisms, fork() and exec()
9. Process Schedulers - Long-term, short-term and medium-term schedulers
10. Multithreading - Benefits and multithreading models
11. Multiprogramming and Multitasking - Comparison
12. Critical Section and Peterson’s Solution - Requirements and algorithm
13. TestAndSet(), Swap(), Race Condition and Busy Waiting - Hardware synchronization mechanisms
14. Bounded-Buffer Producer-Consumer Problem - Semaphore solution and algorithm
15. CPU Scheduling Numerical Problem - Preemptive SJF, priority scheduling and formulas
16. Multiprocessor and Clustered Systems - Architectures and comparison
Part III - Other Operating System Activities
17. OS Activities - Process, memory and file-system management
18. Bootstrap Program and Caching - Definitions, working and comparison
Appendix - Quick Revision - Key terms and scheduling formulae

PART I - OPERATING SYSTEM FOUNDATIONS

1. Operating System Services

Definition: An Operating System (OS) is system software that acts as an interface between users and computer hardware. It manages hardware resources and provides an environment in which application programs execute.

Operating-system services can be grouped into services that make the computer convenient for users and services that help the system operate efficiently.

A. Services for user convenience

1. User Interface

The OS provides an interface through which users interact with the computer. Common interfaces include a Command-Line Interface (CLI), Graphical User Interface (GUI), and touch-based interface. Examples include the Windows desktop and a Linux terminal.

2. Program Execution

The OS loads a program into memory, starts its execution, provides the required resources, and handles normal or abnormal termination.

3. I/O Operations

The OS manages input and output operations involving devices such as keyboards, printers, displays, disks, and network interfaces. Applications use OS interfaces rather than controlling device hardware directly.

4. File-System Manipulation

The OS provides operations to create, delete, open, close, read, write, rename, search, and organize files and directories. It also manages file attributes and permissions.

5. Communication

The OS enables processes to exchange information on the same computer or across a network. Common mechanisms include shared memory, message passing, pipes, and sockets.

6. Error Detection

The OS monitors the system for errors involving the CPU, memory, I/O devices, storage, and programs. It detects errors and takes suitable action to preserve system stability.

B. Services for efficient system operation

7. Resource Allocation

The OS allocates CPU time, main memory, files, storage space, and I/O devices among processes and users. It aims to use available resources efficiently and fairly.

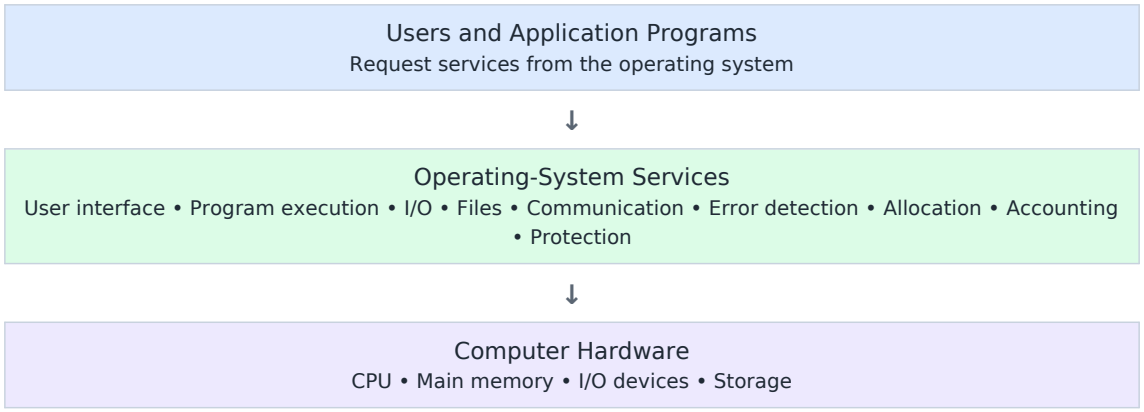
8. Accounting

The OS records resource usage, such as CPU time, memory consumption, and I/O activity. This information can support monitoring, capacity planning, and performance analysis.

9. Protection and Security

Protection controls how processes access system resources. Security prevents unauthorized access through authentication, permissions, and access-control mechanisms.

Diagram: Operating-System Services



The OS provides a service layer between applications and hardware.

Conclusion: Operating-system services make the computer convenient, secure, reliable, and efficient by providing a controlled interface to hardware and managing system resources.

2. Virtual Machines: Concept, Advantages and Challenges

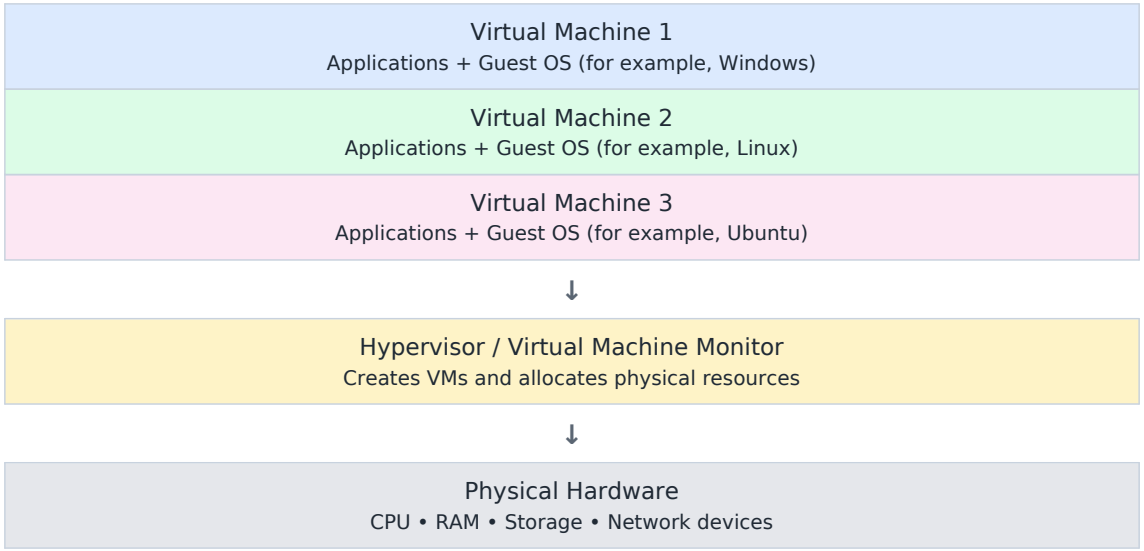
Definition: A Virtual Machine (VM) is a software-based emulation of a computer. It provides virtual CPU, memory, storage, and network resources so that a guest operating system and its applications can run as though they were on a physical computer.

A hypervisor, also called a Virtual Machine Monitor (VMM), creates and manages virtual machines and allocates physical resources to them.

Working of a Virtual Machine

1. The physical computer supplies hardware resources.
2. The hypervisor manages the physical resources and creates virtual machines.
3. Each VM receives virtual CPU, memory, storage, and other devices.
4. Each VM can run its own guest operating system and applications.
5. The hypervisor coordinates access to shared physical resources.

Diagram: Virtual-Machine Architecture



Multiple guest operating systems share one physical computer through the hypervisor.

Types of Hypervisors

1. Type 1 - Bare-Metal Hypervisor

Runs directly on physical hardware without requiring a general-purpose host operating system beneath it. It is commonly used in servers and data centres. Examples: VMware ESXi and Microsoft Hyper-V in its hypervisor role.

2. Type 2 - Hosted Hypervisor

Runs as an application on top of an existing host operating system. It is convenient for desktop testing and development, though it adds an extra software layer. Examples: Oracle VirtualBox and VMware Workstation.

Advantages

- Resource sharing: Several operating systems can run on one physical computer.
- Isolation: Problems in one VM are generally contained and do not automatically affect other VMs.
- Cost reduction: Fewer physical machines may be needed.
- Testing and development: Applications can be tested across different operating systems.
- Portability: A VM can often be copied or moved to compatible hardware.
- Backup and recovery: Snapshots and backups help restore earlier VM states.
- Security: VMs can provide isolated environments for testing untrusted software, when configured properly.

Challenges

- Performance overhead: Virtualization may add overhead compared with direct execution on hardware.
- Resource consumption: Multiple VMs require considerable CPU, RAM, and storage.
- Management complexity: Many VMs need monitoring, patching, and configuration.
- Hardware compatibility: Some specialized devices or software may not work well in virtual environments.
- Security risks: Hypervisor vulnerabilities or incorrect isolation can expose VMs or the host.
- Resource contention: VMs competing for limited resources can reduce performance.

Conclusion: Virtual machines provide flexibility, isolation, and better hardware utilization, but require careful management and sufficient physical resources.

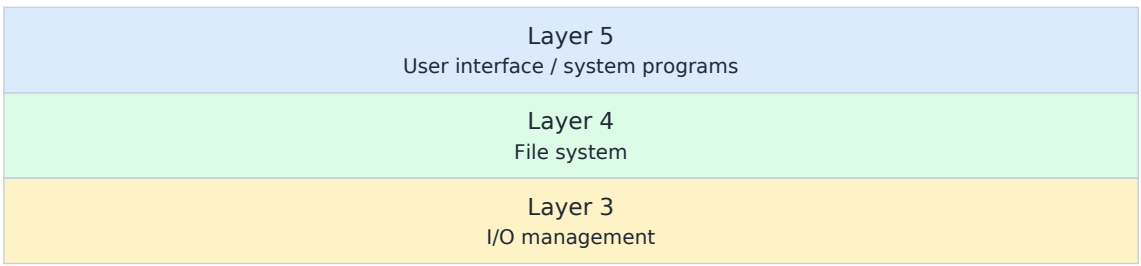
3. Operating-System Structures: Layered, Modular and Microkernel

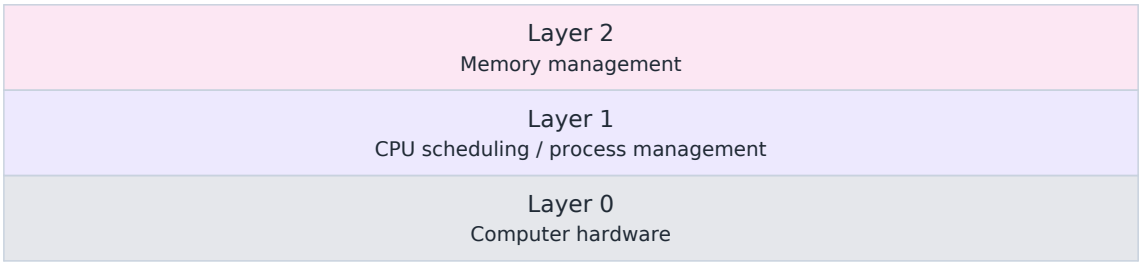
Operating systems use different structures to organize their components. The layered, modular, and microkernel approaches are three important designs.

A. Layered Approach

The OS is divided into hierarchical layers. Each layer performs specific functions and uses services provided by lower layers. The lowest layer is hardware, while higher layers provide increasingly abstract services.

Diagram: Layered Operating System





Each layer uses services of the layer immediately below it.

Advantages of Layered Approach

- Modular organization makes the OS easier to understand.
- Testing and debugging can be performed layer by layer.
- Changes in one layer may be isolated from unrelated layers.
- The structure simplifies maintenance and design.

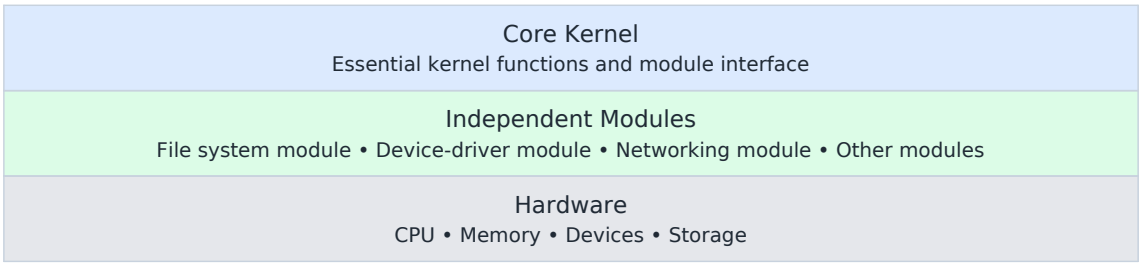
Disadvantages of Layered Approach

- Defining suitable layers and boundaries can be difficult.
- Communication through several layers may add overhead.
- Some operations do not fit neatly into a strict hierarchy.

B. Modular Approach

In a modular OS, the kernel has a core set of functions and can be extended using separate modules. Modules implement services such as device drivers, file systems, and networking. They communicate through defined interfaces; many kernel modules execute in kernel mode.

Diagram: Modular Operating System



Modules extend the core kernel through defined interfaces.

Advantages of Modular Approach

- Modules can be added, removed, or updated independently in supported systems.
- The design is flexible and extensible.
- Well-defined interfaces simplify development and maintenance.
- Kernel services can remain efficient because modules may execute in kernel space.

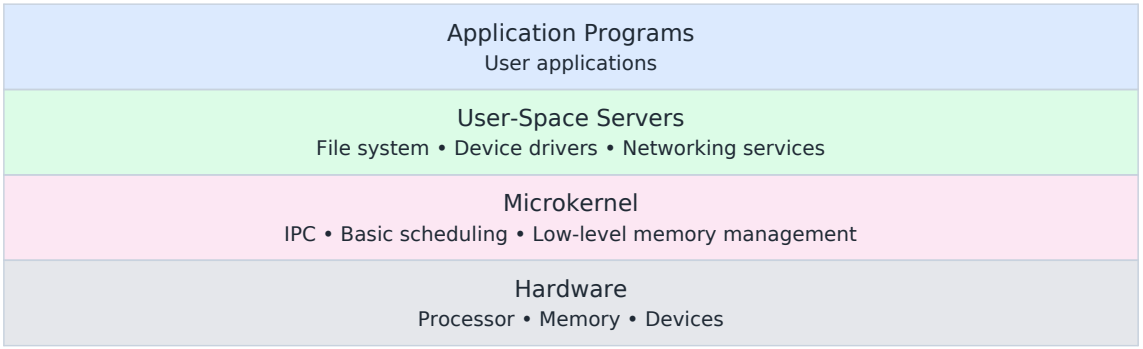
Disadvantages of Modular Approach

- A faulty kernel module can still crash or compromise the kernel.
- Module dependencies and compatibility require careful management.
- The design and debugging of module interactions can be complex.

C. Microkernel Approach

A microkernel keeps only essential mechanisms in kernel mode, such as low-level address-space management, basic scheduling, and inter-process communication. Many services, including file systems and device drivers, run as separate user-space servers.

Diagram: Microkernel Architecture



Many operating-system services run outside the small privileged kernel.

Advantages of Microkernel Approach

- Fault isolation can improve reliability because a failed user-space service need not crash the kernel.
- A smaller privileged kernel can reduce the amount of code that must run with full privileges.
- Services can be maintained or replaced independently.
- The design can improve portability and testability.

Disadvantages of Microkernel Approach

- IPC and context switches between services can add overhead.
- Service coordination and communication can be complex.
- Performance depends on the implementation and workload.

Comparison of Layered, Modular and Microkernel Structures

Layered Structure	Modular Structure	Microkernel Structure
Organizes OS into hierarchical layers.	Organizes OS into independent modules.	Keeps only essential functions in the kernel.
A layer uses services of lower layers.	Modules communicate through defined interfaces.	User-space servers communicate through IPC.
Usually follows a fixed hierarchy.	Modules may be loaded or unloaded in supported systems.	Many services execute as user-space processes.
Provides modularity through layer separation.	Provides modularity through components.	Provides modularity and fault isolation through process separation.
Several layer crossings may add overhead.	In-kernel module calls can be efficient.	IPC and context switching may add overhead.
Example: THE operating system.	Examples: Linux and Solaris use modular kernel designs.	Examples: MINIX 3 and QNX.

Conclusion: Layered architecture emphasizes hierarchy, modular architecture emphasizes replaceable components, and microkernel architecture minimizes the privileged kernel and moves many services into user space.

4. User Mode and Kernel Mode

Modern operating systems use at least two execution modes to protect the system from faulty or unauthorized program operations. A hardware-supported mode bit or privilege level indicates the current mode.

User Mode

User mode is a restricted execution mode in which application programs normally run.

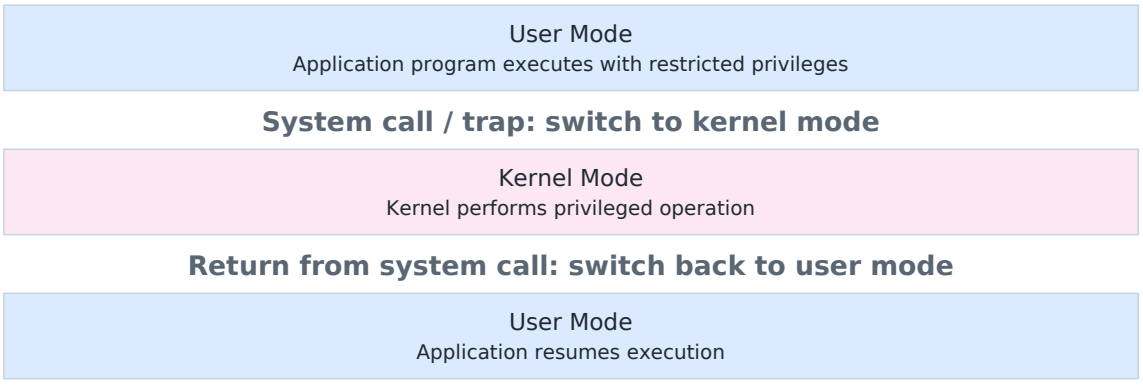
- Applications have limited privileges.
- Direct access to hardware and protected memory is restricted.
- Privileged instructions cannot be executed directly.
- Applications request OS services through system calls.
- An application error is generally contained to the application, though it can still affect other software through bugs or resource exhaustion.

Kernel Mode

Kernel mode is a privileged execution mode used by the OS kernel and other trusted kernel components.

- The kernel can execute privileged instructions.
- It can access hardware and protected memory.
- It handles system calls, interrupts, and exceptions.
- It manages CPU scheduling, memory, devices, and other system resources.
- A kernel failure can affect the entire system.

Diagram: Mode Switching



A system call transfers control into the kernel; return restores user-mode execution.

User Mode	Kernel Mode
Restricted execution mode.	Privileged execution mode.
Runs application programs.	Runs the kernel and trusted privileged components.
Cannot execute privileged instructions.	Can execute privileged instructions.
Access to protected memory and devices is restricted.	Can access protected memory and hardware.
Uses system calls to request OS services.	Handles system calls and provides OS services.
Application failure usually affects the application.	Kernel failure can affect the whole system.

Conclusion: User mode protects the system by restricting applications, while kernel mode allows the operating system to perform privileged operations.

5. System Calls: Role, Types and Implementation

Definition: A system call is a mechanism through which a user-level program requests a service from the operating-system kernel. System calls form a controlled interface between applications and the kernel.

Role of System Calls

- Process management: Create, execute, wait for, and terminate processes. Examples: `fork()`, `exec()`, `wait()`, `exit()`.
- File management: Create, open, read, write, close, and delete files. Examples: `open()`, `read()`, `write()`, `close()`.

- Device management: Request device operations and control device attributes. Examples: `read()`, `write()`, `ioctl()`.
- Memory management: Map, unmap, or adjust a process memory region. Examples on Unix-like systems include `mmap()`, `munmap()`, and `brk()`.
- Information maintenance: Obtain process or system information. Examples: `getpid()`, `time()`, `uname()`.
- Communication: Exchange data using pipes, sockets, shared memory, and related facilities. Examples: `pipe()`, `send()`, `recv()`.
- Protection: Manage permissions and identities. Examples: `chmod()`, `chown()`, `setuid()`.

Six Traditional Categories of System Calls

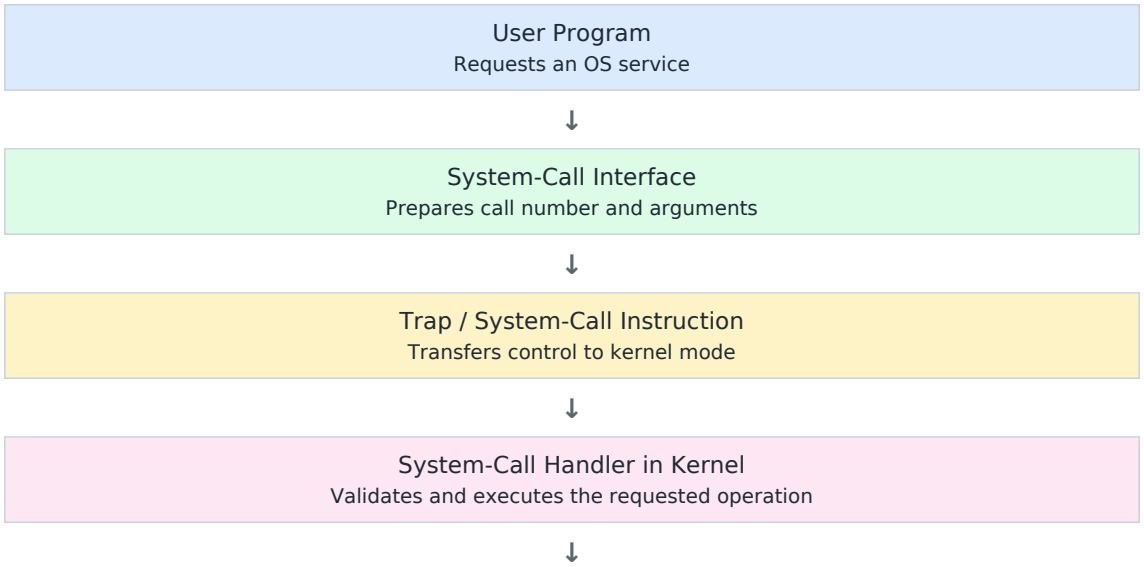
Category	Purpose	Examples
Process control	Create, execute, wait for, and terminate processes.	fork(), exec(), exit(), wait()
File management	Create, open, read, write, close, and delete files.	open(), read(), write(), close()
Device management	Request and control I/O devices.	read(), write(), ioctl()
Information maintenance	Get or set system and process information.	getpid(), time(), uname()
Communication	Exchange information between processes.	pipe(), send(), recv()
Protection	Control access and permissions.	chmod(), chown(), setuid()

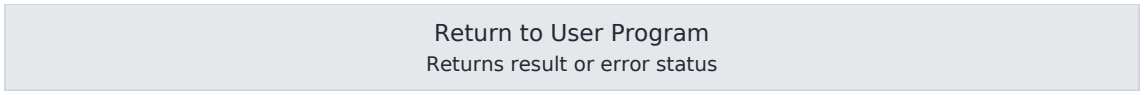
Implementation of System Calls

A system call is normally invoked through a library wrapper or system-call interface. The interface arranges the system-call number and arguments, executes a controlled trap or system-call instruction, and transfers control to the kernel.

1. A user program requests an OS service, for example, reading a file.
2. A library wrapper or system-call interface prepares the system-call number and arguments.
3. A trap or system-call instruction transfers control to a predefined kernel entry point.
4. The processor enters kernel mode and the system-call handler identifies the requested operation.
5. The kernel validates arguments and permissions, then executes the corresponding kernel routine.
6. The kernel places a return value or error status where the calling program can obtain it.
7. Control returns to user mode and the application continues.

Diagram: System-Call Implementation





System calls use a controlled transition from user mode into kernel mode and back.

Methods of Passing Parameters to System Calls

1. Registers: Parameters are placed directly in CPU registers. This is fast but limited by the number and size of available registers.
2. Memory block or table: Parameters are stored in a memory block, and the address of that block is passed in a register.
3. Stack: The program pushes parameters onto the stack, from which the kernel or entry mechanism retrieves them according to the calling convention.

Advantages / Importance

- Provide controlled access to privileged OS services.
- Protect hardware and kernel data from direct application access.
- Offer a standard programming interface to system resources.
- Support resource management, communication, file access, and process control.

Conclusion: System calls connect user applications with the kernel. Their implementation uses a system-call interface, call number and arguments, a controlled trap into kernel mode, a kernel handler, and a return to user mode.

6. Inter-Process Communication: Shared Memory and Message Passing

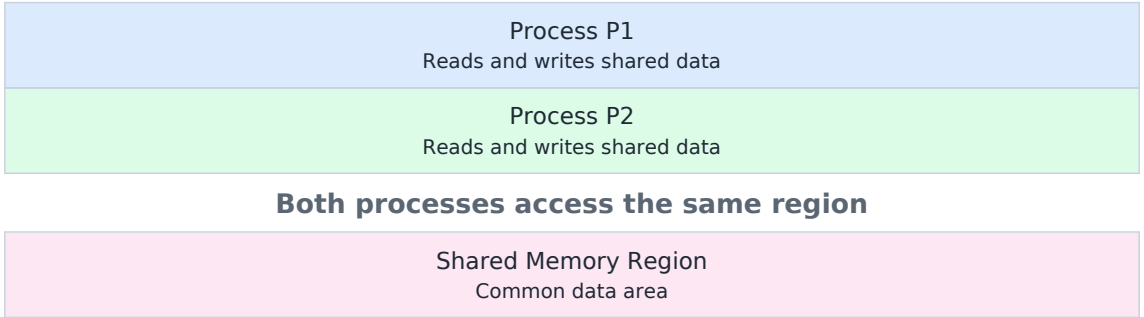
Definition: Inter-Process Communication (IPC) is a set of mechanisms that allows processes to exchange information and coordinate their activities. Two fundamental IPC approaches are shared memory and message passing.

A. Shared Memory

In shared-memory IPC, the operating system establishes a memory region that is mapped into the address spaces of two or more processes. The processes communicate by reading and writing data in this common region.

1. The OS creates or identifies a shared memory region.
2. The region is mapped into the address spaces of the participating processes.
3. Processes read and write data directly in the shared region.
4. Processes use synchronization tools such as mutexes or semaphores to prevent conflicting access.

Diagram: Shared-Memory IPC



A shared region is accessible to both processes; synchronization is still required.

Advantages

- Fast communication after the region is established.
- Suitable for large amounts of data.
- Reduces repeated message copying.
- Useful for processes on the same computer.

Disadvantages

- Processes must coordinate access explicitly.
- Race conditions and inconsistent data can occur without synchronization.
- Programming and debugging can be more difficult.

B. Message Passing

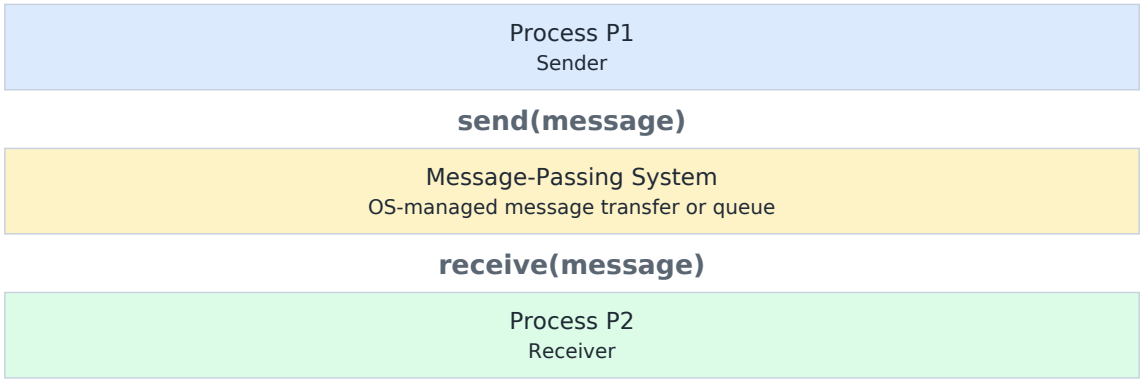
In message-passing IPC, processes communicate by sending and receiving messages. The OS or communication subsystem manages the transfer between processes.

Basic message-passing operations

```
send(message)
receive(message)
```

1. The sender prepares a message.
2. The sender invokes a send operation.
3. The OS or communication mechanism transfers or queues the message.
4. The receiver invokes a receive operation and obtains the message.
5. The receiver processes the received data.

Diagram: Message-Passing IPC



Processes exchange messages without directly sharing an address-space region.

Advantages

- Processes remain isolated in their own address spaces.
- The communication mechanism can provide synchronization.
- Can be used locally or across a network.
- Often easier to structure as separate services.

Disadvantages

- May be slower for large data transfers because messages can require copying.
- Communication involves the OS or messaging subsystem.
- Message size, queue capacity, and buffering can limit performance.

Comparison

Shared Memory	Message Passing
Uses a common memory region.	Uses send and receive operations.
Fast for large data transfers after setup.	May have more overhead for large data transfers.
Requires explicit synchronization.	Synchronization may be built into the communication mechanism.
Processes access the same region.	Processes retain separate address spaces.
Mainly used on the same machine.	Can be used locally or over a network.
Application programmers manage shared-data consistency.	OS or communication subsystem manages message transfer.

Conclusion: Shared memory offers high-speed communication through a common region, while message passing exchanges data through messages and provides stronger separation between processes.

PART II - PROCESSES AND CPU MANAGEMENT

7. Process, Process States and Process Control Block (PCB)

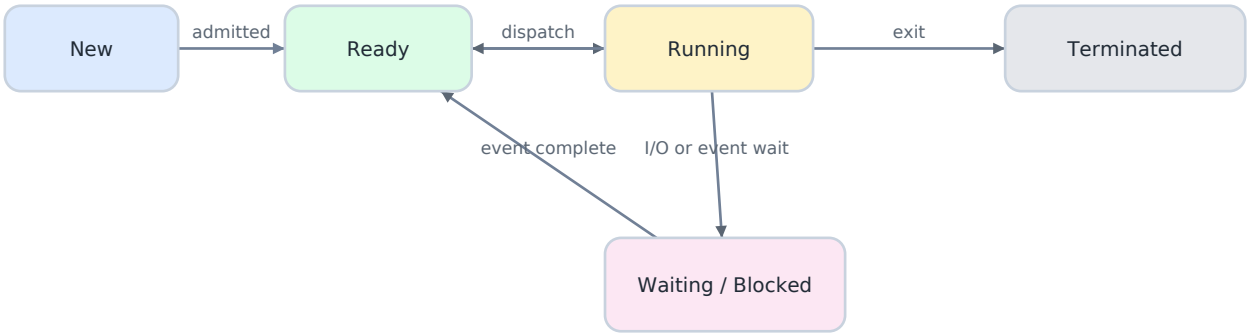
Definition: A process is a program in execution. A program is a passive file stored on secondary storage, while a process is an active entity with execution state and allocated resources.

A process generally includes a text (code) section, data section, heap, stack, program counter, and CPU register contents.

Five Basic Process States

1. New: The process is being created. The OS creates its PCB and performs initial setup.
2. Ready: The process is in main memory and is waiting to be assigned the CPU.
3. Running: The process instructions are currently executing on a CPU.
4. Waiting / Blocked: The process is waiting for an event, such as completion of an I/O operation.
5. Terminated: The process has finished or has been stopped, and its resources are being reclaimed.

Diagram: Process-State Transitions



A process moves between states as it is admitted, scheduled, blocked, resumed, or terminated.

Process-State Transitions

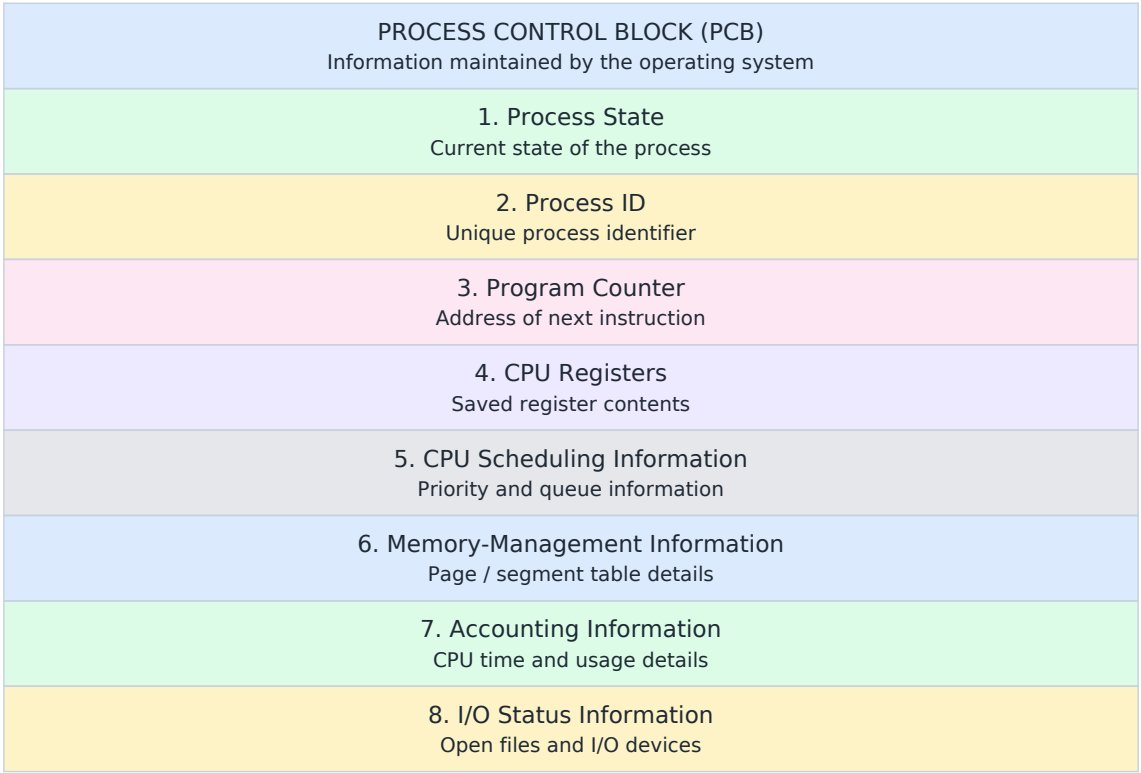
Transition	Meaning
New → Ready	The process is admitted to the ready queue.
Ready → Running	The CPU scheduler dispatches the process.
Running → Ready	The process is preempted or its time slice expires.
Running → Waiting	The process requests I/O or waits for an event.
Waiting → Ready	The I/O operation or event completes.
Running → Terminated	The process completes or exits.

Process Control Block (PCB)

Definition: A Process Control Block is a kernel data structure that stores the information required to manage and resume a process. Each process has an associated PCB.

1. Process state: New, ready, running, waiting, or terminated.
2. Process ID (PID): Unique identifier assigned to the process.
3. Program counter: Address of the next instruction to execute.
4. CPU registers: Register values saved during a context switch and restored when the process resumes.
5. CPU-scheduling information: Priority, scheduling parameters, and queue pointers.
6. Memory-management information: Page-table, segment-table, or other address-space information.
7. Accounting information: CPU time used, process or user identifiers, and resource usage.
8. I/O status information: Open files, allocated devices, and pending I/O information.

Diagram: Process Control Block



The PCB enables the OS to track a process and perform context switching.

Conclusion: A process is an executing program. The OS tracks its progress through process states and stores its management information in the PCB.

8. Process Creation and Process Termination; fork() and exec()

A. Process Creation

Definition: Process creation is the mechanism by which the OS creates a new process to execute a program. The process that creates another process is called the parent, and the new process is called the child.

Mechanisms / Steps of Process Creation

1. Assign a PID: The OS assigns a unique process identifier.
2. Create the PCB: The OS creates a Process Control Block containing the process state, program counter, registers, and scheduling information.
3. Allocate resources: The OS provides or arranges memory and other resources.
4. Create an address space: The child receives its own logical process address space. In UNIX-like systems, this may initially use copy-on-write sharing with the parent.
5. Initialize process information: The OS sets up initial execution and management information.
6. Place in a scheduling queue: The process is made eligible for scheduling when it is ready.

fork() System Call

In UNIX-like systems, `fork()` creates a child process. After a successful call, both parent and child continue execution at the instruction following `fork()`. The child initially has a logical copy of the parent address space, often implemented with copy-on-write.

fork() return values

```
pid_t pid = fork();

if (pid == 0) {
    /* child process */
} else if (pid > 0) {
    /* parent process; pid is child PID */
} else {
    /* process creation failed */
}
```

- Returns `0` in the child process.
- Returns the child PID (a positive value) in the parent process.
- Returns `-1` on failure.

exec() Family

The `exec()` family replaces the current process image with a new program. The process identity (PID) normally remains the same. On success, the old program does not resume; the call returns only if an error occurs.

Example of execvp()

```
char *args[] = {"ls", "-l", NULL};
execvp("ls", args);
/* Reached only if execvp fails. */
```

fork() and exec() Comparison

fork()	exec()
Creates a new child process.	Replaces the current process program.
Parent and child both exist after success.	The same process runs the new program.
Child receives a logical copy of the parent address space.	Current process address space is replaced.
Returns in both parent and child.	Does not return on success.
Child has a different PID from parent.	PID normally remains unchanged.
Used to create a process.	Used to load and execute another program.

Other Process-Creation Interfaces

Windows provides `CreateProcess()` to create a new process and its primary thread. Other operating systems provide their own process-creation APIs.

B. Process Termination

Definition: Process termination is the mechanism through which the OS stops a process and reclaims the resources allocated to it.

Types / Mechanisms of Process Termination

1. Normal termination: The process completes successfully and calls `exit()`.
2. Abnormal termination: The process is stopped because of an error such as invalid memory access, illegal instruction, or another fatal exception.
3. Termination by another process: Another process or the OS may send a termination signal or request. On Unix-like systems, `kill()` sends a signal; whether it terminates the target depends on the signal and handling.
4. Parent termination: If a parent exits while children remain, the OS follows its process policy; in Unix-like systems, orphaned children are typically reparented.
5. Cascading termination: Some systems or policies terminate descendants when a parent is terminated.

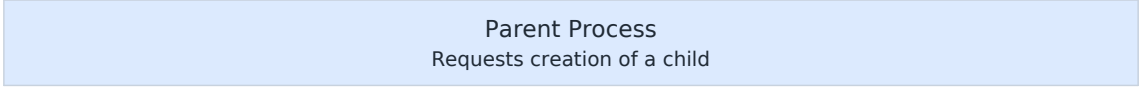
Steps in Process Termination

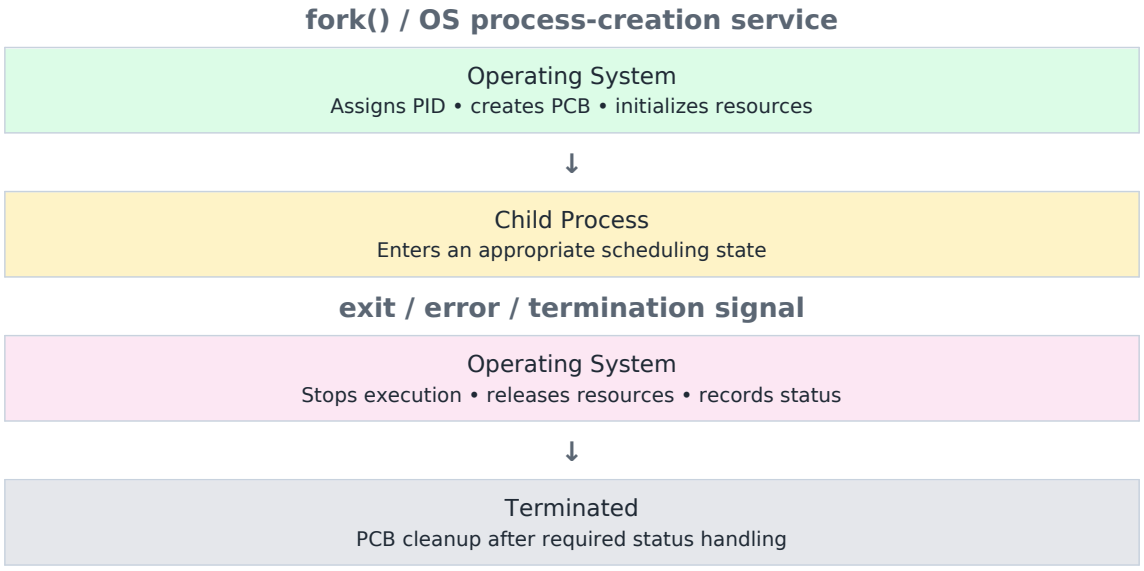
1. Stop the process execution.
2. Release memory and other allocated resources.
3. Close or release open files and devices as appropriate.
4. Update accounting and exit-status information.
5. Notify the parent or relevant process manager.
6. Remove the PCB after required cleanup and status collection.

Important Termination / Waiting Calls

Call	Purpose
exit()	Terminates the calling process normally.
_exit()	Terminates a process directly at the system-call level on Unix-like systems.
abort()	Causes abnormal process termination.
kill()	Sends a signal to a process; signal semantics determine the result.
wait() / waitpid()	Allows a parent to wait for a child and collect its termination status.

Diagram: Process Creation and Termination





The OS manages both the creation and cleanup stages of a process life cycle.

Conclusion: Process creation establishes a new execution context and its PCB; process termination stops execution, records the outcome, and releases resources. In Unix-like systems, `fork()` creates a child and `exec()` replaces a process image.

9. Long-Term, Short-Term and Medium-Term Schedulers

Definition: A scheduler is an OS component that selects processes from queues and determines which processes enter memory, receive the CPU, or are swapped out.

1. Long-Term Scheduler (Job Scheduler)

Selects jobs from the job pool on secondary storage and admits selected processes into main memory / the ready queue. It controls the degree of multiprogramming and can help maintain a suitable mix of CPU-bound and I/O-bound jobs.

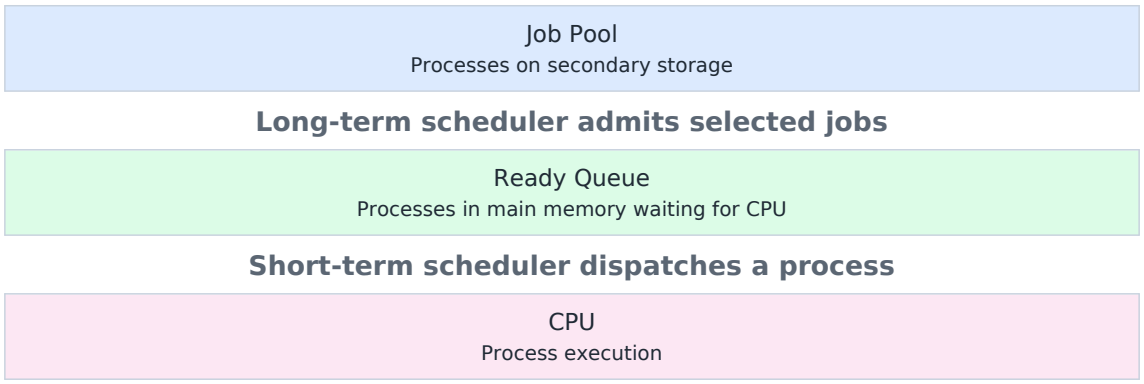
2. Short-Term Scheduler (CPU Scheduler)

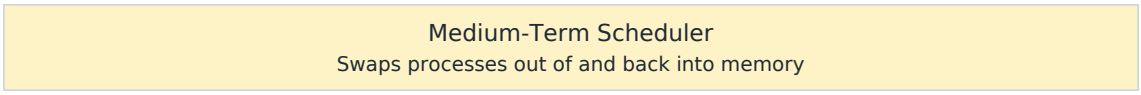
Selects a process from the ready queue and allocates the CPU to it. It runs very frequently and therefore must be fast.

3. Medium-Term Scheduler (Swapper)

Temporarily removes processes from main memory and later restores them. Swapping can reduce memory pressure and adjust the degree of multiprogramming.

Diagram: Process Scheduling





The long-term scheduler admits jobs, the short-term scheduler selects for the CPU, and the medium-term scheduler manages swapping.

Long-Term Scheduler	Short-Term Scheduler	Medium-Term Scheduler
Also called job scheduler.	Also called CPU scheduler.	Also called swapper.
Selects jobs from job pool.	Selects processes from ready queue.	Selects processes for swapping out or in.
Admits processes into memory.	Allocates the CPU.	Temporarily removes and restores processes.
Controls degree of multiprogramming at admission.	Controls CPU allocation.	Helps control memory pressure and multiprogramming.
Runs relatively infrequently.	Runs very frequently.	Runs when swapping is required.
Can be relatively slow.	Must be very fast.	Frequency depends on system conditions.
Balances the mix of admitted jobs.	Improves CPU use and response.	Improves memory availability.

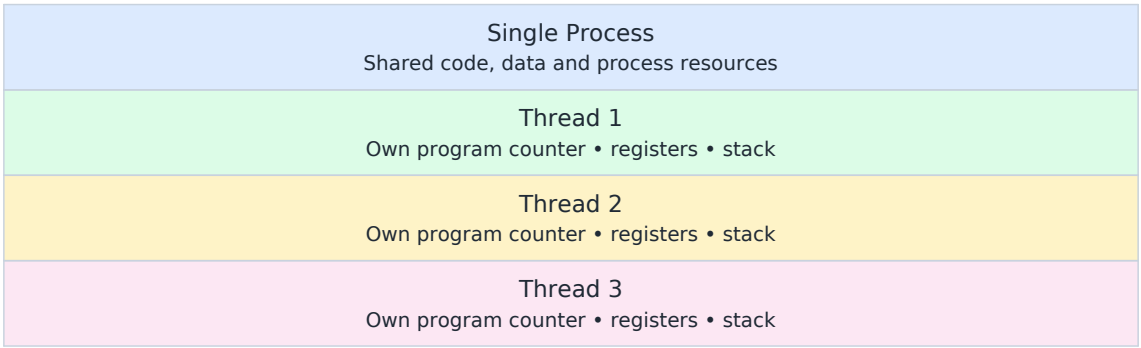
Conclusion: The long-term scheduler controls job admission, the short-term scheduler chooses the next process for the CPU, and the medium-term scheduler manages process swapping.

10. Multithreading: Definition, Benefits and Models

Definition: Multithreading is the ability of a process to contain multiple threads of execution. A thread is the smallest unit of CPU execution. Threads in the same process share code, data, and many process resources, but each thread has its own program counter, register set, and stack.

Example: A browser may use separate threads for responding to user input, loading a page, downloading data, and playing media.

Diagram: Threads Within One Process



Threads share process resources but maintain their own execution state.

Benefits of Multithreading

1. Responsiveness: An application can remain responsive while another thread performs a lengthy operation.
2. Resource sharing: Threads of a process share code, data, and open resources, simplifying communication.
3. Economy: Creating and managing threads is generally less costly than creating and managing separate processes.
4. Scalability: Multiple threads can run on different CPU cores in a multiprocessor system.
5. Better CPU utilization: When one thread waits for I/O, another runnable thread may use the CPU.

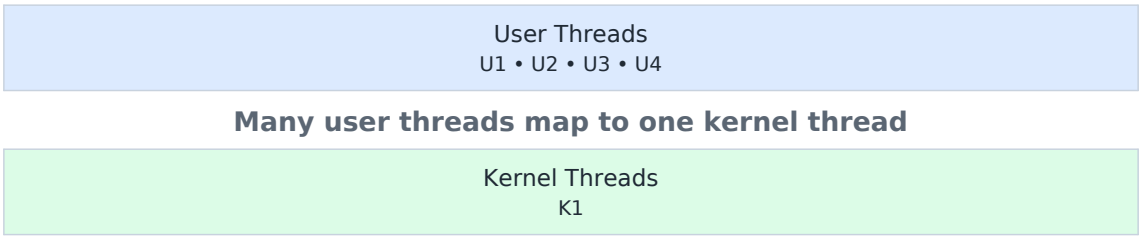
Thread-Mapping Models

Multithreading models describe how user-level threads are mapped to kernel-level threads.

A. Many-to-One Model

Many user threads are mapped to one kernel thread. Thread management can be fast and mostly in user space, but a blocking system call can block the process, and the model cannot achieve parallel execution through multiple kernel threads.

Diagram: Many-to-One



Many user threads share a single kernel thread.

Advantages

- Fast user-level thread management.
- Low kernel-management overhead.

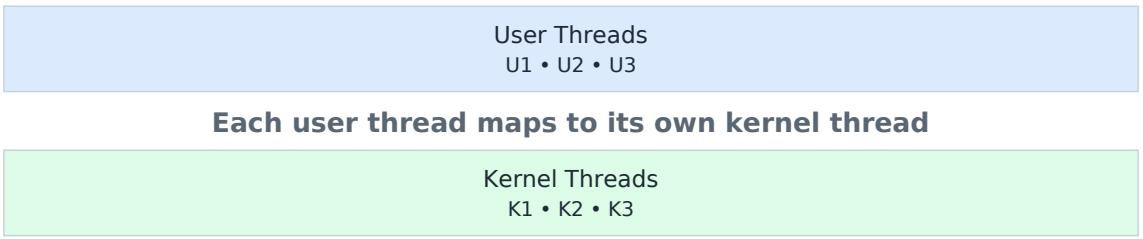
Disadvantages

- A blocking operation may block all threads in the process.
- No parallel execution across multiple kernel threads.

B. One-to-One Model

Each user thread maps to a separate kernel thread. The model permits true parallelism on multiple cores and allows other threads to run when one blocks, but it uses more kernel resources.

Diagram: One-to-One



Every user thread has a corresponding kernel thread.

Advantages

- Supports parallel execution on multiple CPU cores.
- Blocking one thread need not block the other threads.

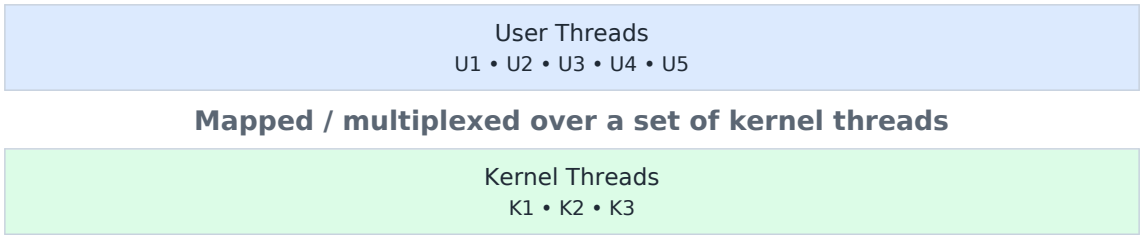
Disadvantages

- Creating a user thread requires a kernel thread.
- Large thread counts can increase kernel resource and scheduling overhead.

C. Many-to-Many Model

Many user threads are multiplexed over a set of kernel threads, where the number of kernel threads can be smaller than or equal to the number of user threads. This combines flexible user-level thread management with kernel-supported concurrency.

Diagram: Many-to-Many



Multiple user threads are mapped over multiple kernel threads.

Advantages

- Supports concurrency and parallelism.
- Can support many user threads without requiring an equal number of kernel threads.
- One blocked thread does not necessarily block all others.

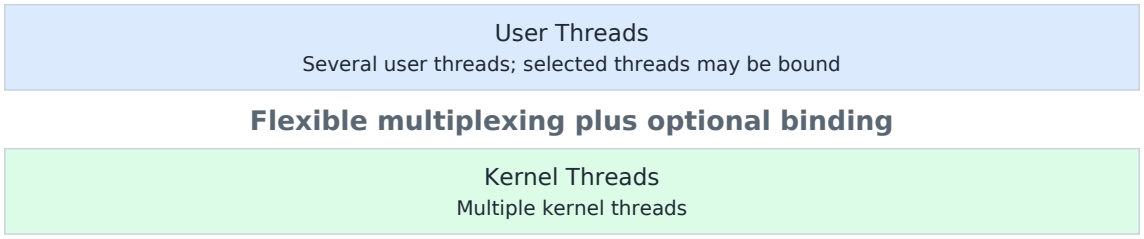
Disadvantages

- Implementation and coordination are complex.
- Mapping and scheduling require careful design.

D. Two-Level Model

The two-level model is a variation of many-to-many. It permits many user threads to be mapped over multiple kernel threads while allowing selected user threads to be bound to particular kernel threads.

Diagram: Two-Level Model



Two-level model adds optional binding to the many-to-many approach.

Comparison of Multithreading Models

Many-to-One	One-to-One	Many-to-Many
Many user threads → one kernel thread.	One user thread → one kernel thread.	Many user threads → multiple kernel threads.
Limited concurrency.	Kernel-level concurrency.	Flexible concurrency.
Cannot run multiple user threads in parallel on multiple kernel threads.	Supports parallel execution.	Supports parallel execution.
A blocking system call can block the process.	One blocked thread need not block others.	Other mapped threads can continue if resources permit.
Low kernel overhead.	Higher kernel resource usage.	More complex management.
Example: some older green-thread implementations.	Examples: Windows and modern Linux threading implementations.	Historically supported by systems such as Solaris.

Conclusion: Multithreading improves responsiveness, sharing, economy, and scalability. The models differ in how user threads are mapped to kernel threads.

11. Multiprogramming and Multitasking Systems

Multiprogramming: Several programs are kept in main memory. When one program waits for I/O, the OS can run another so that the CPU remains busy.

Multitasking: The CPU rapidly switches among multiple tasks, commonly using time slices and preemption, to provide interactive execution and the appearance of simultaneous activity.

Multiprogramming	Multitasking
Multiple programs are kept in main memory.	Multiple tasks or processes share CPU time.
CPU can switch when the current job waits for I/O.	CPU commonly switches on time-slice expiry, preemption, or blocking.
Main aim is to maximize CPU utilization.	Main aim is interactive responsiveness and time-sharing.
Traditionally associated with batch systems.	Common in interactive and general-purpose systems.
User interaction is limited in traditional batch systems.	Designed to support user interaction with multiple tasks.
Response time is not usually the main goal.	Quick response is an important goal.
Focuses on keeping the CPU busy.	Focuses on sharing CPU time among tasks.

Example: In multiprogramming, if Program A waits for disk input, the CPU runs Program B. In multitasking, a user can browse the web, listen to music, and edit a document while the OS rapidly schedules their tasks.

Conclusion: Multiprogramming primarily improves CPU utilization by overlapping computation and I/O, while multitasking provides time-shared, responsive execution for multiple tasks.

12. Critical-Section Problem and Peterson’s Solution

Critical section: A critical section is the part of a process or thread that accesses shared data or a shared resource. The critical-section problem is to design a protocol that coordinates access so concurrent execution does not corrupt shared state.

General Structure of a Process

```
do {
    // Entry section

    // Critical section

    // Exit section

    // Remainder section
} while (true);
```

Three Requirements of a Correct Solution

1. Mutual Exclusion: At most one process can be executing in its critical section for a given shared resource at a time.
2. Progress: If no process is in the critical section and some wish to enter, the choice of the next process must not be postponed indefinitely. Only processes that want to enter should participate in the choice.
3. Bounded Waiting: After a process requests entry, there must be a bound on how many times other processes can enter before its request is granted. This prevents indefinite postponement.

Peterson’s Solution

Peterson’s solution is a classic software algorithm for two cooperating processes, P0 and P1. It uses two shared variables: `flag[2]` indicates interest in entering the critical section, and `turn` resolves a simultaneous request.

Peterson’s algorithm for two processes

```
/* Shared initialization */
boolean flag[2] = { false, false };
int turn;

/* Process P0 */
do {
    flag[0] = true;
```

```
    turn = 1;
    while (flag[1] && turn == 1) { }

    /* Critical section */

    flag[0] = false;
    /* Remainder section */
} while (true);

/* Process P1 */
do {
    flag[1] = true;
    turn = 0;
    while (flag[0] && turn == 0) { }

    /* Critical section */

    flag[1] = false;
    /* Remainder section */
} while (true);
```

Working

1. When P0 wants to enter, it sets `flag[0] = true` and sets `turn = 1`.
2. P0 waits while P1 is interested and it is P1's turn.
3. P1 performs the symmetric operations with `flag[1]` and `turn`.
4. When a process leaves the critical section, it clears its flag.
5. If both processes request entry together, the `turn` variable resolves which one waits.

Requirement	How Peterson's Solution Meets It
Mutual exclusion	The flag and turn protocol prevents both processes from entering together.
Progress	If the critical section is free, the protocol allows one interested process to proceed.
Bounded waiting	A waiting process cannot be bypassed indefinitely by the other process.

Advantages and Limitations

- It is a simple software solution for two processes.
- It demonstrates mutual exclusion, progress, and bounded waiting.
- It uses busy waiting.
- The classic proof assumes atomic reads and writes and a suitable memory model. Modern implementations need appropriate atomic operations and memory-ordering guarantees.
- The basic algorithm is for two processes, not an arbitrary number.

Conclusion: Peterson's solution uses `flag[]` and `turn` to coordinate two processes and meet the three critical-section requirements under its assumptions.

13. TestAndSet(), Swap(), Race Condition and Busy Waiting

TestAndSet() and Swap() are atomic hardware-supported instructions that can be used to implement mutual exclusion. An atomic operation completes indivisibly with respect to competing processors or threads.

A. TestAndSet() Instruction

TestAndSet() returns the old value of a lock variable and sets the lock to true as one atomic operation.

TestAndSet-based lock (illustrative pseudocode)

```
boolean TestAndSet(boolean *target) {
    boolean old_value = *target;
    *target = true;
    return old_value;
```

```
}

/* Initial value */
boolean lock = false;

do {
    while (TestAndSet(&lock)) {
        ;  /* busy waiting */
    }

    /* Critical section */

    lock = false;
    /* Remainder section */
} while (true);
```

Working of TestAndSet()

1. Initially, `lock = false`.
2. A process executes TestAndSet atomically.
3. The instruction returns the old value and sets the lock to true.
4. If the old value was false, the process acquires the lock and enters the critical section.
5. If the old value was true, the process keeps waiting.
6. On leaving the critical section, the process sets the lock to false.

B. Swap() Instruction

Swap() atomically exchanges the values of two variables.

Swap-based lock (illustrative pseudocode)

```
void Swap(boolean *a, boolean *b) {
    boolean temp = *a;
    *a = *b;
    *b = temp;
}

/* Initial shared lock */
boolean lock = false;

do {
    boolean key = true;
    while (key == true) {
        Swap(&lock, &key);
    }

    /* Critical section */

    lock = false;
    /* Remainder section */
} while (true);
```

Working of Swap()

1. Initially, `lock = false`.
2. A process initializes its local variable `key = true`.
3. The atomic Swap operation exchanges `lock` and key.
4. If the old lock was false, the process receives false in key and enters the critical section.
5. If the old lock was true, key remains true and the process continues waiting.
6. The process releases the lock by setting it to false after leaving the critical section.

Comparison: TestAndSet() and Swap()

TestAndSet()	Swap()
Reads the old value of a target and sets it to true.	Exchanges the values of two variables.
Returns the previous value of the target.	Performs an atomic exchange.
Often uses one shared lock variable.	Uses a shared lock and a local key variable in the shown algorithm.
Can implement a spinlock.	Can implement mutual exclusion through atomic exchange.
Requires hardware-supported atomicity.	Requires hardware-supported atomicity.

C. Race Condition

Definition: A race condition occurs when multiple processes or threads access and update shared data concurrently, and the final result depends on the timing or ordering of their operations.

Example: Let shared `count = 5`. Two processes both execute `count = count + 1`. If both read 5 before either writes, both may write 6. The expected result is 7, but the actual result becomes 6. The increment is a read-modify-write sequence and is not atomic.

Step	Process P1	Process P2	Count
1	Reads 5		5
2		Reads 5	5
3	Calculates 5 + 1		5
4		Calculates 5 + 1	5
5	Writes 6		6
6		Writes 6	6

Prevention of Race Conditions

- Mutual-exclusion locks and mutexes.
- Semaphores and monitors.
- Atomic operations.
- Correctly designed critical sections and synchronization protocols.

D. Busy Waiting

Definition: Busy waiting occurs when a process repeatedly checks a condition or lock instead of sleeping or blocking while it waits for a resource.

```
while (lock == true) {
    /* keep checking the lock */
}
```

Advantages: Simple to implement and can be useful when waiting is expected to be very short, because it avoids a sleep/wakeup context switch.

Disadvantages: It wastes CPU cycles while waiting and can reduce efficiency when the wait is long or contention is high.

Conclusion: TestAndSet() and Swap() provide atomic primitives for locks. Race conditions arise from unsynchronized concurrent access, while busy waiting consumes CPU time by repeatedly checking a condition.

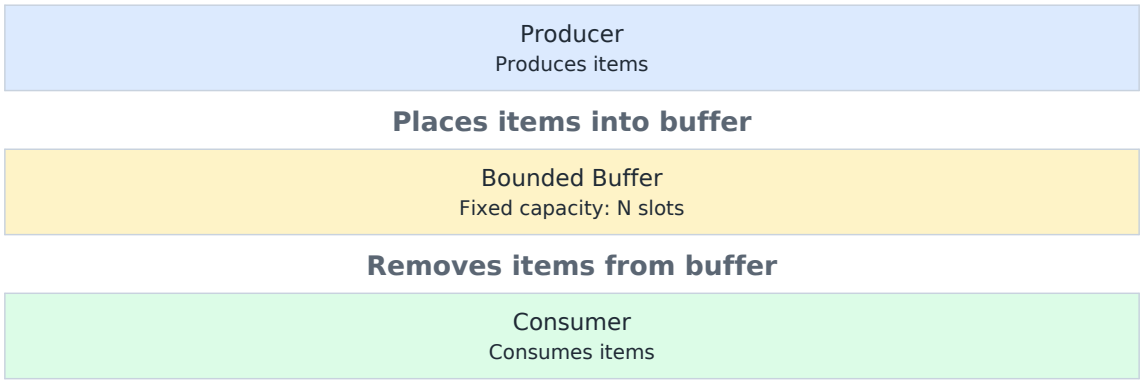
14. Bounded-Buffer Producer-Consumer Problem

Definition: The Producer-Consumer Problem is a classical synchronization problem. A producer creates items and places them into a shared buffer, while a consumer removes and uses those items. In the bounded-buffer version, the buffer has a fixed capacity N.

Conditions

- The producer must not insert an item when the buffer is full.
- The consumer must not remove an item when the buffer is empty.
- Only one process should modify the shared buffer at a time.

Diagram: Bounded Buffer



The producer inserts items and the consumer removes them from a fixed-size buffer.

Semaphore Solution

Three semaphores are used: `mutex` protects the buffer, `empty` counts empty slots, and `full` counts occupied slots.

Semaphore	Initial Value	Purpose
mutex	1	Ensures exclusive access to the shared buffer.
empty	N	Counts empty buffer slots.
full	0	Counts occupied buffer slots.

wait(S) and signal(S) are atomic semaphore operations. A process calling wait on an unavailable resource blocks until it can proceed; signal increments the semaphore and may wake a waiting process.

Producer Algorithm

```
do {
    // Produce an item

    wait(empty);
    wait(mutex);

    // Add item to buffer

    signal(mutex);
    signal(full);
} while (true);
```

Explanation of Producer

1. Produce an item.
2. Call `wait(empty)` to ensure that an empty slot exists.
3. Call `wait(mutex)` to obtain exclusive access to the buffer.
4. Insert the item into the buffer.
5. Call `signal(mutex)` to release the buffer lock.

6. Call `signal(full)` to indicate that one more item is available.

Consumer Algorithm

```
do {
    wait(full);
    wait(mutex);

    // Remove item from buffer

    signal(mutex);
    signal(empty);

    // Consume the item
} while (true);
```

Explanation of Consumer

1. Call `wait(full)` to ensure an item is available.
2. Call `wait(mutex)` to obtain exclusive access to the buffer.
3. Remove an item from the buffer.
4. Call `signal(mutex)` to release the lock.
5. Call `signal(empty)` to indicate that one more slot is free.
6. Consume the removed item.

How the Algorithm Solves the Problem

- Mutual exclusion: mutex allows only one process to modify the buffer at a time.
- No overflow: the producer waits when empty is zero, meaning the buffer is full.
- No underflow: the consumer waits when full is zero, meaning the buffer is empty.
- Progress: when a required condition is satisfied, the corresponding process can proceed. Proper semaphore queueing can provide bounded waiting.

Conclusion: The bounded-buffer problem uses mutex, empty, and full semaphores to coordinate producers and consumers, avoid buffer overflow and underflow, and protect the shared buffer.

15. CPU Scheduling Numerical Problem: Preemptive SJF and Preemptive Priority

> Important data note: The process table referenced in the original question was not included in the chat. Therefore, no exact Gantt chart or numerical average can be calculated without inventing arrival times, CPU burst times, and priorities. This section gives the VTU solving procedure and calculation format.

Given Data Required

Process	Arrival Time (AT)	CPU Burst Time (BT)	Priority
P1	Required	Required	Required
P2	Required	Required	Required
P3	Required	Required	Required
P4	Required	Required	Required

(i) Preemptive Shortest Job First - Shortest Remaining Time First (SRTF)

Definition: The CPU is allocated to the arrived process with the smallest remaining CPU burst time. If a new process arrives with a smaller remaining time than the currently running process, the current process is preempted.

Steps

1. Write arrival time and burst time for every process.
2. At each time instant, consider only processes that have arrived and are not complete.
3. Select the process with the least remaining burst time.
4. When a new process arrives, compare its burst time with the currently running process's remaining time. Preempt if the new process has the smaller remaining time.
5. Continue until all processes complete, then record each completion time.
6. Draw the Gantt chart and calculate turnaround and waiting times.

Illustrative Gantt-chart format only - not the solution to the missing data

```
|  P1  | P2 |  P3  | P2 | P1 |
0      3    5      8    10   12
```

(ii) Preemptive Priority Scheduling

Definition: The CPU is allocated to the arrived process with the highest priority. In this question, a smaller priority number indicates a higher priority. A newly arrived higher-priority process preempts the running process.

Steps

1. Write arrival time, CPU burst time, and priority for every process.
2. At each time instant, consider all arrived and incomplete processes.
3. Select the process with the smallest priority number.
4. If a higher-priority process arrives, preempt the current process.
5. Continue until all processes complete and record completion times.
6. Draw the Gantt chart and calculate the required averages.

Formulae

Turnaround Time (TAT): the total time from arrival to completion.

```
TAT = Completion Time (CT) - Arrival Time (AT)
```

Waiting Time (WT): the total time spent waiting in the ready queue.

```
WT = TAT - CPU Burst Time (BT)
   = CT - AT - BT
```

Average Waiting Time:

```
Average WT = Sum of all waiting times / Number of processes
```

Average Turnaround Time:

```
Average TAT = Sum of all turnaround times / Number of processes
```

Calculation Table Format

Process	AT	BT	CT	TAT = CT - AT	WT = TAT - BT
P1	-	-	-	-	-
P2	-	-	-	-	-
P3	-	-	-	-	-
P4	-	-	-	-	-
Total				ΣTAT	ΣWT

Conclusion: Preemptive SJF selects the process with the shortest remaining time, while preemptive priority selects the arrived process with the smallest priority number. Completion times obtained from the Gantt chart are used to calculate waiting and turnaround times.

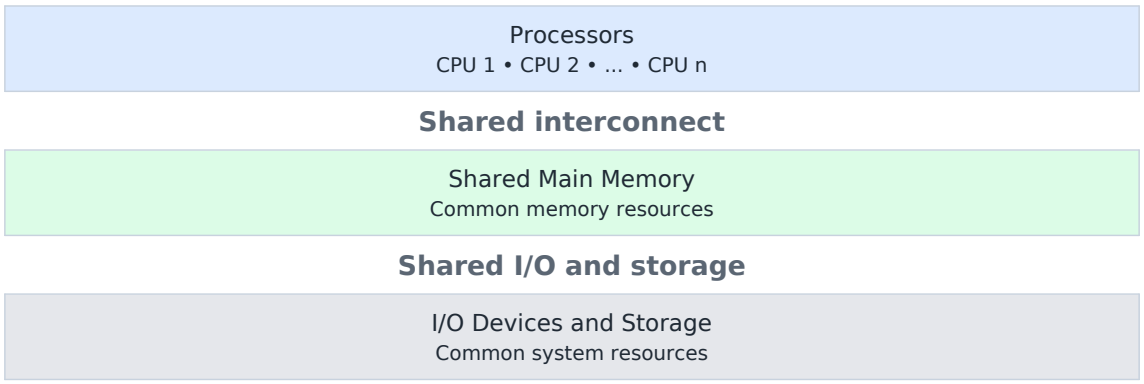
16. Multiprocessor Systems and Clustered Systems

Multiprocessor system: A computer system with two or more processors that share system resources such as main memory, storage, and I/O devices. Processors may execute tasks in parallel.

Clustered system: A group of independent computers (nodes) connected through a network and coordinated to work together. Clustered systems are often designed for high availability, scalability, or parallel workloads.

Multiprocessor Architecture

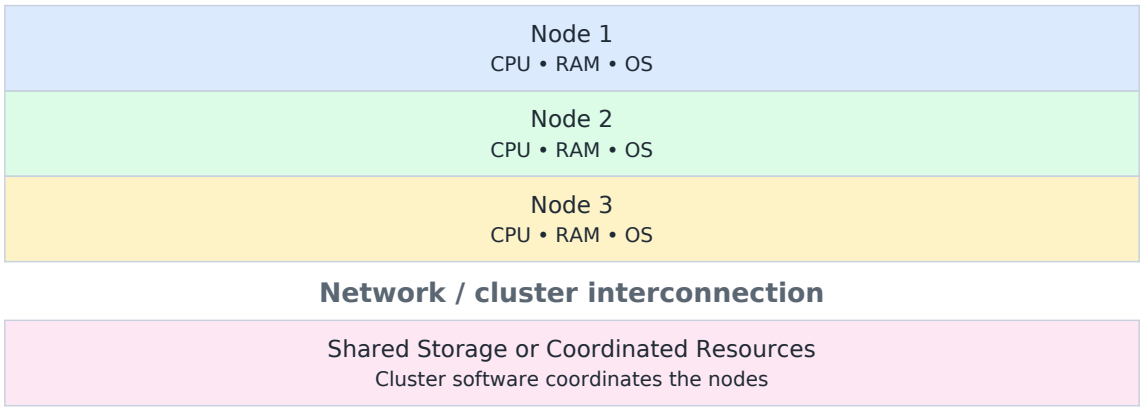
Diagram: Multiprocessor System



Multiple processors operate within one computer system and share resources.

Clustered Architecture

Diagram: Clustered System



Independent computers communicate over a network and may share storage or coordinate resources.

Types of Multiprocessor Systems

Asymmetric Multiprocessing (AMP): One processor acts as a master and assigns work to other processors, which may have specialized roles.

Symmetric Multiprocessing (SMP): Processors are peers and can run OS and user tasks, sharing memory and I/O resources.

Types of Clustered Systems

Asymmetric clustering: One machine runs applications actively while another remains on standby and can take over if the active machine fails.

Symmetric clustering: Two or more machines run applications simultaneously and monitor one another, improving utilization and availability.

Multiprocessor Systems	Clustered Systems
Two or more processors in one computer.	Two or more independent computers (nodes).
Processors share memory and I/O resources.	Nodes communicate through a network.
Usually one OS instance manages the system.	Each node generally runs its own OS instance.
Communication often uses shared memory or a hardware interconnect.	Communication uses network protocols and cluster software.
Tightly coupled architecture.	Loosely coupled architecture.
Primarily increases parallel processing and throughput.	Can provide high availability, scalability, and parallel processing.
A processor failure may be tolerated depending on system design.	A node failure can be handled by another node in a fault-tolerant cluster.
Example: multi-core or multi-socket server.	Example: high-availability server cluster.

Advantages

Multiprocessor: Higher computational throughput, parallel execution, shared-resource efficiency, and potentially improved reliability.

Clustered: High availability, scalability by adding nodes, fault tolerance, and load distribution across machines.

Conclusion: Multiprocessor systems combine processors within one computer, while clustered systems connect independent computers to work together.

PART III - OTHER OPERATING-SYSTEM ACTIVITIES

17. OS Activities Related to Process Management, Memory Management and File-System Management

An operating system manages hardware and software resources to support efficient program execution. Three major areas of OS activity are process management, memory management, and file-system management.

A. Process Management

Definition: Process management deals with the creation, execution, scheduling, synchronization, communication, and termination of processes.

1. Process creation and deletion: Creates new processes and terminates completed or stopped processes.
2. Process scheduling: Selects processes from scheduling queues and allocates CPU time.
3. Process suspension and resumption: Temporarily suspends and later resumes processes when required.
4. Process synchronization: Coordinates access to shared resources using mechanisms such as mutexes and semaphores.
5. Process communication: Provides IPC mechanisms such as shared memory and message passing.
6. Deadlock handling: Uses prevention, avoidance, detection, and recovery strategies as appropriate to the system.

Summary of Process-Management Activities

- Creating and deleting processes.
- Scheduling processes and allocating CPU time.
- Suspending and resuming processes.

- Providing synchronization and communication.
- Handling deadlocks.

B. Memory Management

Definition: Memory management tracks and controls main memory, allocates memory to processes, and reclaims it when no longer needed.

1. Tracking memory usage: Records which memory regions are in use and which are free.
2. Memory allocation and deallocation: Allocates memory to processes and releases it when no longer required.
3. Deciding what remains in memory: Determines which processes or pages should be resident, swapped, or brought into memory.
4. Virtual-memory management: Supports execution when the working set does not fit entirely in RAM, using techniques such as paging.
5. Memory protection: Prevents unauthorized access to another process's address space.
6. Memory sharing: Allows selected regions to be shared, for example for IPC or shared libraries.

Summary of Memory-Management Activities

- Track used and free memory.
- Allocate and deallocate memory.
- Manage virtual memory and page movement.
- Provide memory protection.
- Support controlled memory sharing.

C. File-System Management

Definition: File-system management creates, deletes, organizes, stores, retrieves, and protects files and directories on secondary storage such as HDDs and SSDs.

1. File creation and deletion: Creates new files and removes unwanted files.
2. Directory creation and deletion: Organizes files into directories and removes directories when appropriate.
3. File operations: Supports open, close, read, write, rename, search, and related operations.
4. Mapping to secondary storage: Maps files and file blocks to storage locations and tracks allocation.
5. Backup and recovery: Supports backup tools and recovery mechanisms, depending on the OS and configuration.
6. File protection and permissions: Controls read, write, and execute access for users and processes.

Summary of File-System Activities

- Create and delete files and directories.
- Provide file manipulation operations.
- Map files onto secondary storage.
- Manage disk-space allocation.
- Support backup and recovery.
- Enforce access permissions.

Comparison of the Three OS Activities

Process Management	Memory Management	File-System Management
Manages processes.	Manages main memory.	Manages files and directories.
Creates and deletes processes.	Allocates and deallocates memory.	Creates and deletes files.
Performs CPU scheduling.	Tracks used and free memory.	Organizes files and directories.
Provides synchronization and IPC.	Provides memory protection and virtual memory.	Provides file protection and permissions.
Handles process coordination and deadlocks.	Supports memory sharing and allocation.	Maps files to secondary storage and manages space.

Conclusion: Process management controls execution, memory management allocates and protects RAM, and file-system management organizes and protects stored data. Together they support efficient and reliable computer operation.

18. Bootstrap Program and Caching

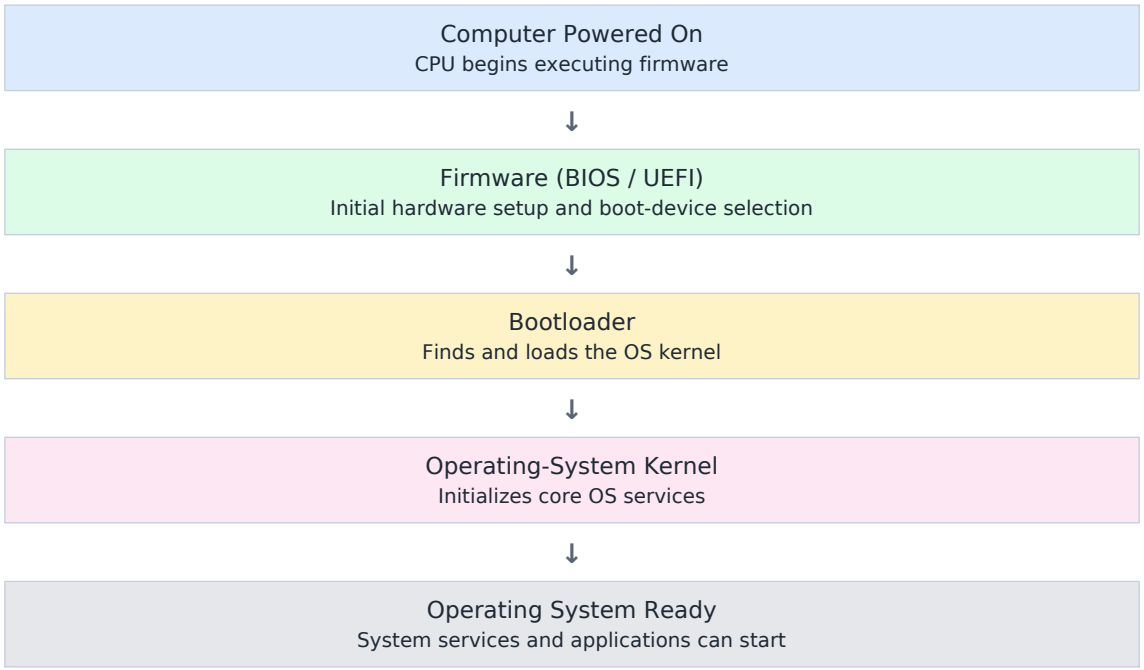
A. Bootstrap Program

Definition: A bootstrap program is a small initialization program that runs when a computer is powered on or reset. It performs initial system setup and begins loading the operating system.

Working of Bootstrap Program

1. When the computer is powered on, the processor begins executing firmware instructions.
2. The firmware performs basic hardware initialization and checks.
3. The boot process identifies a bootable device or boot manager.
4. The bootloader locates and loads the operating-system kernel, or an earlier boot stage that leads to the kernel load.
5. Control is transferred to the kernel, which continues system initialization.

Diagram: Boot Process



The firmware and bootloader cooperate to load and start the OS kernel.

Example: BIOS or UEFI firmware initiates the boot process. A bootloader such as GRUB may then load the Linux kernel.

Conclusion: The bootstrap process initializes the system and starts loading the operating-system kernel so that the computer can begin normal operation.

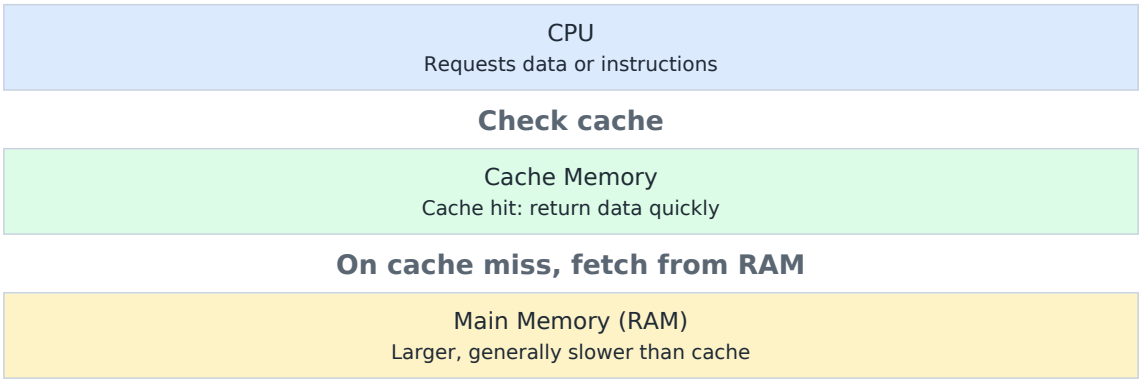
B. Caching

Definition: Caching is a technique in which copies of frequently or recently accessed data are kept in a faster storage location so that future accesses can be served more quickly. In a CPU memory hierarchy, cache memory is a small, high-speed memory between CPU execution units and main memory.

Working of Caching

1. The CPU or system requests data.
2. The system checks the cache for the requested data.
3. If the data is present, it is a cache hit and the data is supplied from the cache.
4. If the data is absent, it is a cache miss; the data is fetched from a lower-level memory, such as RAM.
5. A copy may be placed in cache so that a later access can be faster.

Diagram: CPU Cache and Main Memory



A cache hit avoids a slower access to the next level of the memory hierarchy.

Cache Levels

1. L1 cache: Usually the smallest and fastest CPU cache, commonly private to a core.
2. L2 cache: Usually larger and somewhat slower than L1.
3. L3 cache: Usually larger than L1 and L2 and often shared among multiple CPU cores.

Advantages of Caching

- Reduces average data-access time when the cache hit rate is high.
- Reduces repeated accesses to slower memory or storage.
- Can improve CPU and application performance.
- Uses locality of reference: recently used or nearby data is often accessed again.

Example: A CPU may keep recently used instructions and data in cache so that repeated accesses do not need to go to RAM every time.

Conclusion: Caching stores copies of useful data in faster storage to reduce access latency and improve performance.

Bootstrap Program vs Caching

Bootstrap Program	Caching
Starts or initiates the computer boot process.	Improves data-access performance.
Initializes hardware and begins loading the OS kernel.	Stores copies of frequently or recently accessed data.
Runs during startup or reset.	Operates during normal system activity.
Uses firmware and bootloader stages.	Uses a faster storage layer such as CPU cache.
Purpose is to start the OS.	Purpose is to reduce average access time.

APPENDIX - QUICK REVISION

Key Terms

Topic	Remember
Operating System	Interface between users/applications and hardware.
Virtual Machine	Software-based computer environment managed by a hypervisor.
Layered OS	Hierarchical layers; each uses lower-layer services.
Modular OS	Core kernel extended by modules through defined interfaces.
Microkernel	Small privileged kernel; many services run in user space.
User Mode	Restricted execution mode for applications.
Kernel Mode	Privileged mode for OS kernel operations.
System Call	Controlled request from a user program to the kernel.
IPC	Mechanisms for processes to communicate.
Process	Program in execution.
PCB	Kernel data structure containing process-management information.
fork()	Creates a child process in Unix-like systems.
exec()	Replaces the current process image with a new program.
Long-Term Scheduler	Admits jobs into memory.
Short-Term Scheduler	Selects a process for the CPU.
Medium-Term Scheduler	Swaps processes out and back into memory.
Multithreading	Multiple threads within one process.
Critical Section	Code that accesses shared data or resources.
Peterson’s Solution	Two-process software synchronization using flag[] and turn.
TestAndSet()	Atomic operation that returns old value and sets target.
Swap()	Atomic exchange of two values.
Race Condition	Concurrent result depends on execution order.
Busy Waiting	Repeatedly checking a condition while waiting.
Producer-Consumer	Bounded-buffer synchronization using mutex, empty, full.
Multiprocessor	Multiple processors in one computer.
Cluster	Multiple independent computers coordinated over a network.
Bootstrap	Startup mechanism that initializes and loads the OS.
Caching	Stores useful copies in faster storage to reduce access time.

CPU Scheduling Formula Sheet

```
Turnaround Time (TAT) = Completion Time (CT) - Arrival Time (AT)

Waiting Time (WT) = Turnaround Time (TAT) - Burst Time (BT)
                  = CT - AT - BT

Average Waiting Time (AWT) = Sum of WT / Number of processes

Average Turnaround Time (ATAT) = Sum of TAT / Number of processes
```

Last-Minute Comparison Reminders

- Multiprogramming: keep the CPU busy by switching when a job waits; Multitasking: share CPU time for interactive responsiveness.
- Layered: hierarchy; Modular: independent components; Microkernel: minimal kernel plus user-space servers.
- Long-term: admission; Short-term: CPU selection; Medium-term: swapping.
- Multiprocessor: several processors in one system; Cluster: several independent networked systems.
- Shared memory: common region; Message passing: send/receive messages.
- Many-to-One: many user threads to one kernel thread; One-to-One: one each; Many-to-Many: many user threads over multiple kernel threads.

Numerical-question reminder: For the preemptive SJF and preemptive priority problem, always use the process table provided in the question. The arrival time, burst time, and priority values are essential; a numerical result must not be guessed.